

1. Github Commit History

Link: https://github.com/suprice4/UniSell_repo#

Commit Hash	Commit Date	Commit Message	Description
25cd25f	July 7, 2026	Add category management UI to vendor dashboard (Android)	Created and configured the Category list, UI Add, Edit, and Delete in the Android vendor dashboard using a RecyclerView adapter and interfaced with the newly created Category API layer.
89fff40	July 8, 2026	Implement Category CRUD backend: entity, repository, service, controller with vendor ownership enforcement	Created the entire back-end functionality for category management, including the Category entity, database table, a vendor-scoped repository, a service layer to validate the name and enforce per-vendor ownership, and REST controller endpoints for create/list/update/delete .
8d68258	July 8, 2026	Add category management UI to vendor dashboard (Web)	Designed and implemented Category list UI, including add, edit inline, and delete categories, for the ReactJS vendor dashboard, replacing the previous placeholders.

5289eee	July 8, 2026	Add Product entity (SRS Feature 1, WIP - repository, service, and controller not yet implemented)	Introduced the Product JPA entity, added name, sku, price, quantity, low-stock threshold, and relationships to the other entities of category and vendor owning the product, preparing the ground for the next feature in Product Management.
---------	--------------	---	---

Commits

main
All users
All time

Commits on Jul 8, 2026

Add Product entity (SRS Feature 1, WIP - repository, service, and controller not yet implemented) 5289eee
suprice4 committed 19 minutes ago

Add category management UI to vendor dashboard (Web) 8d68258
suprice4 committed 20 minutes ago

Implement Category CRUD backend: entity, repository, service, controller with vendor ownership enforcement 89fff40
suprice4 committed 20 minutes ago

Commits on Jul 7, 2026

Add category management UI to vendor dashboard (Android) 25cd25f
suprice4 committed 9 hours ago

2. Implementation Explanation Report

Feature: Category Management (Vendor)

Purpose: Provides a vendor with the ability to establish, view, rename, and eliminate their own product categories, which will classify products in the future Product Management feature (SRS Feature 1). Strictly isolates data; no vendor can see or edit any other vendor's categories.

ReactJS component: VendorDashboard.jsx — list view with add form, inline edit, and delete with confirmation.

Android screen: VendorDashboardActivity.kt: CategoryAdapter.kt (RecyclerView-based list), add/edit dialogs, delete confirmation.

The components of Spring Boot that are used:

- **Category.java (Entity)** - belongs to the categories table, has a foreign key relationship to the owning vendor
- **CategoryRepository.java (Repository)** - JPA data access, vendor-scoped queries
- **CategoryRequest.java / CategoryResponse.java (DTOs)** - shape for the API request/response.

- **CategoryService.java (Service)** - business logic: ownership enforcement (extracts the authenticated vendor's identity from the security context that is populated by the JWT, not from client input).
- **CategoryController(Controller)** - Provides the REST endpoints.

API endpoints and HTTP methods:

Method	Endpoint	Purpose
POST	/api/categories	Create a new category
GET	/api/categories	List the authenticated vendor's categories
PUT	/api/categories/{id}	Update a category's name
DELETE	/api/categories/{id}	Delete a category

Database table: categories (Supabase PostgreSQL), with a vendor_id foreign key referencing users.

Flow from user action to database result:

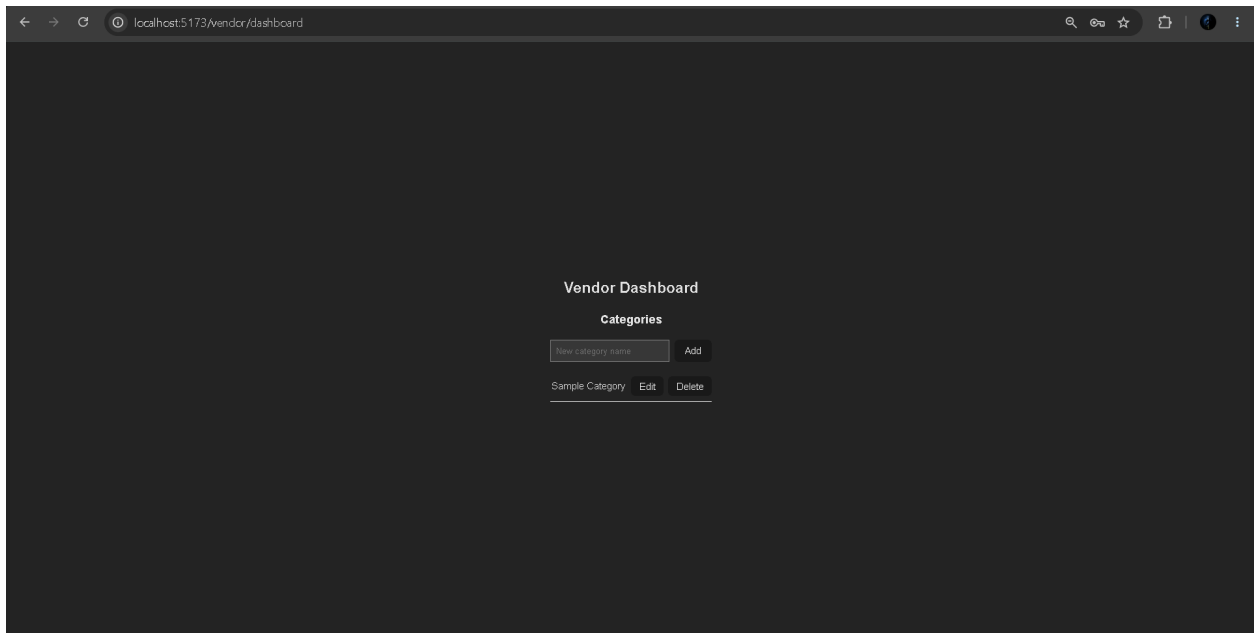
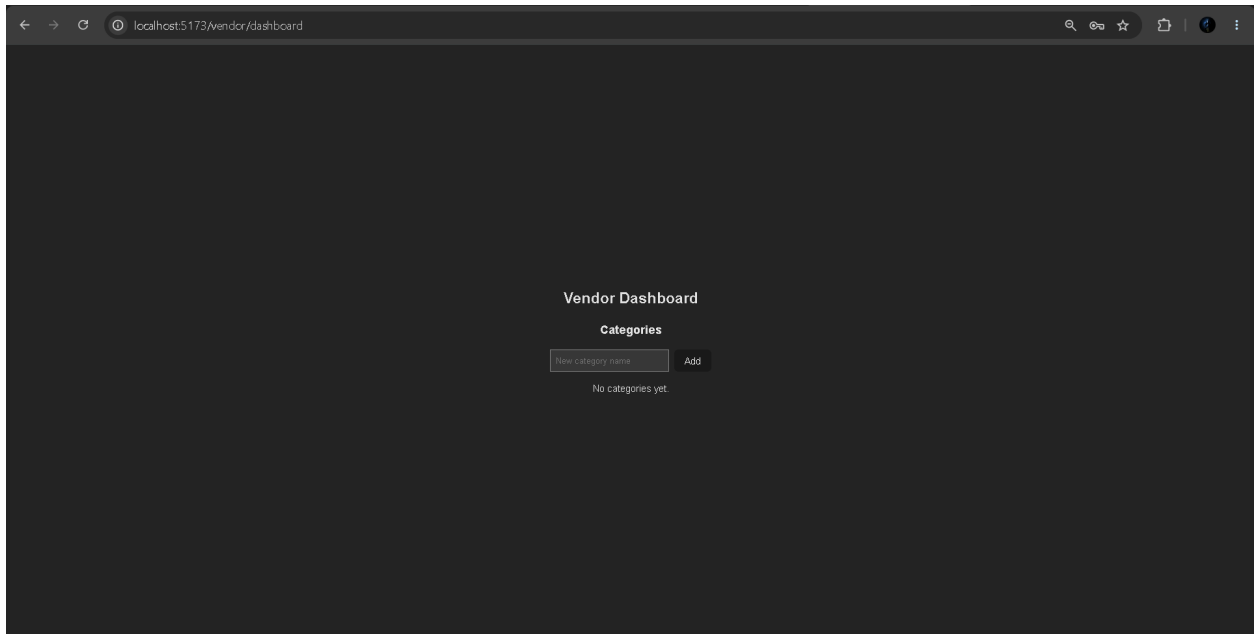
- Vendor logs in and is given a JWT (email + role claims, HS256-signed).
- This token is automatically attached to any request as Authorization: Bearer header on the frontend (axios.js) and Android (ApiClient.kt) and automatically updated when the response returns it as X-New-Token with a sliding time of 30 minutes following the expiration time, as required by SRS FR-021.
- A POST /api/categories request is made when the vendor fills out the "add category" form.
- JwtAuthFilter checks the token for validity and sets the vendor's identity and role into Spring Security's context.
- The request goes to CategoryController, which delegates to CategoryService.
- CategoryService gets the authenticated vendor from the security context (not from client-provided ID), checks that the category name is non-empty and not a duplicate for that vendor, and stores a new row in CategoryRepository.
- This saved category is returned to the client and shown in the list.
- The same ownership check occurs when a user tries to perform an action on a category owned by another user, such as when a user tries to delete a category that another user created: If it does not exist, it doesn't come back as 404, but as 403.

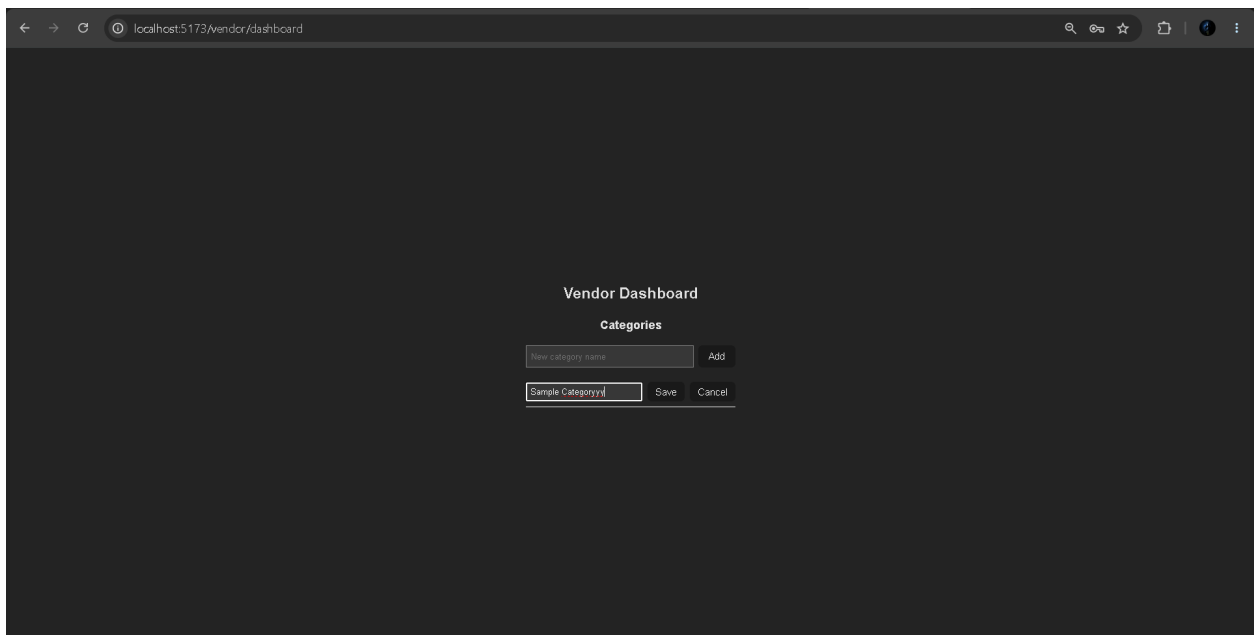
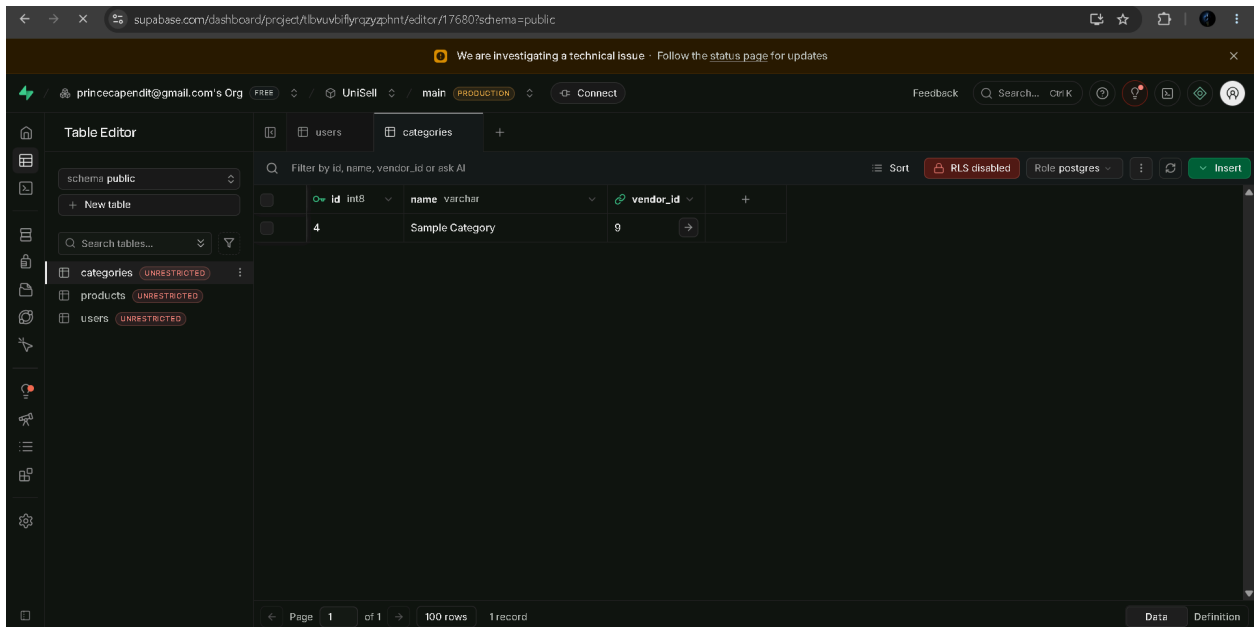
Important source files and responsibilities:

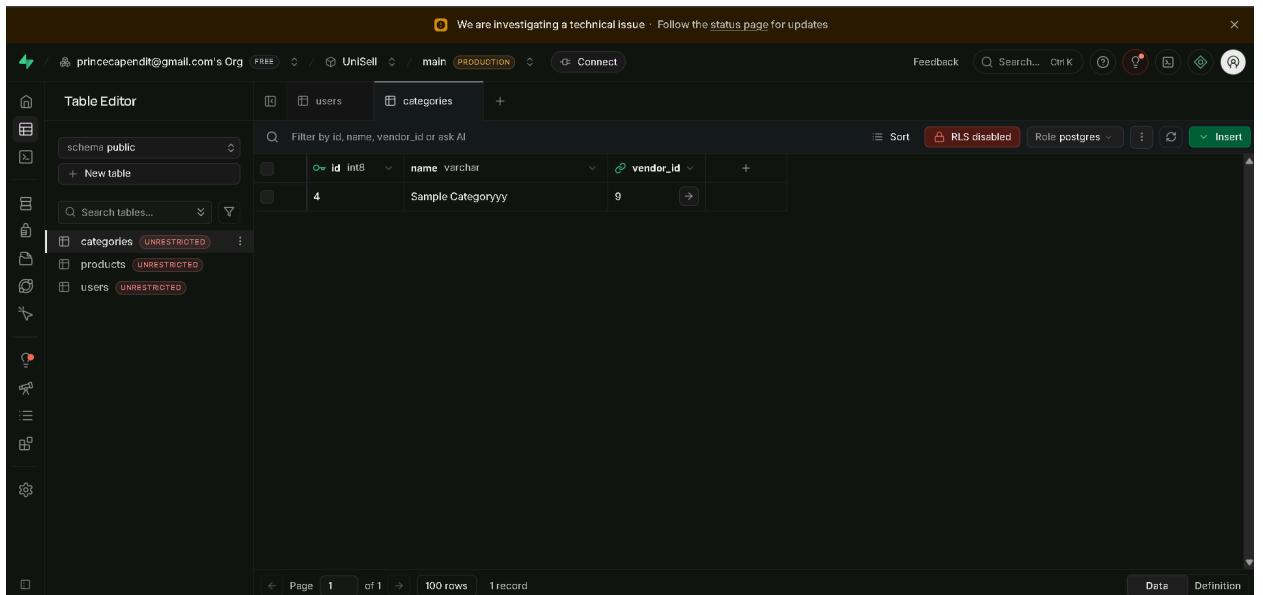
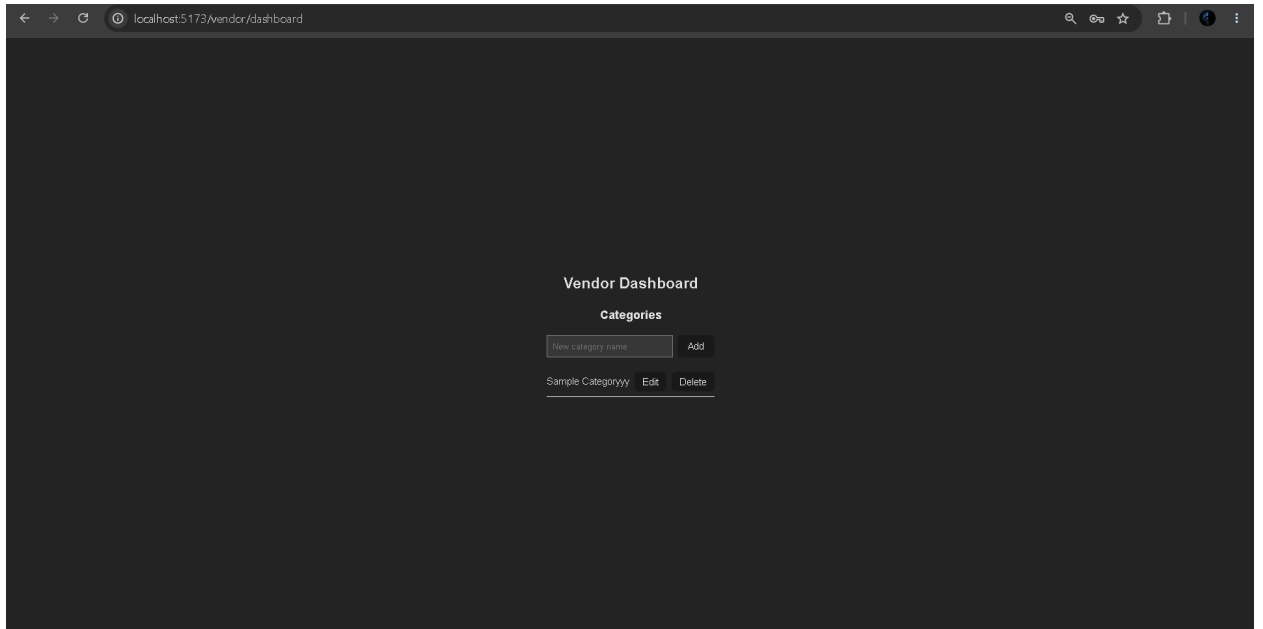
- **CategoryController.java** - HTTP request handling, delegates to service
- **CategoryService.java** - ownership enforcement, validation rules
- **CategoryRepository.java** - vendor-scoped database queries
- **web/src/pages/VendorDashboard.jsx** - web UI (list, add, inline edit, delete)

- **web/src/api/axios.js** - automatic JWT attach/refresh (shared across all authenticated requests)
- **mobile/.../VendorDashboardActivity.kt** - Android UI (list, add/edit dialogs, delete confirmation)
- **mobile/.../ApiClient.kt** - automatic JWT attach/refresh for Android (mirrors axios.js)
- **mobile/.../CategoryAdapter.kt** - RecyclerView list rendering and row interactions

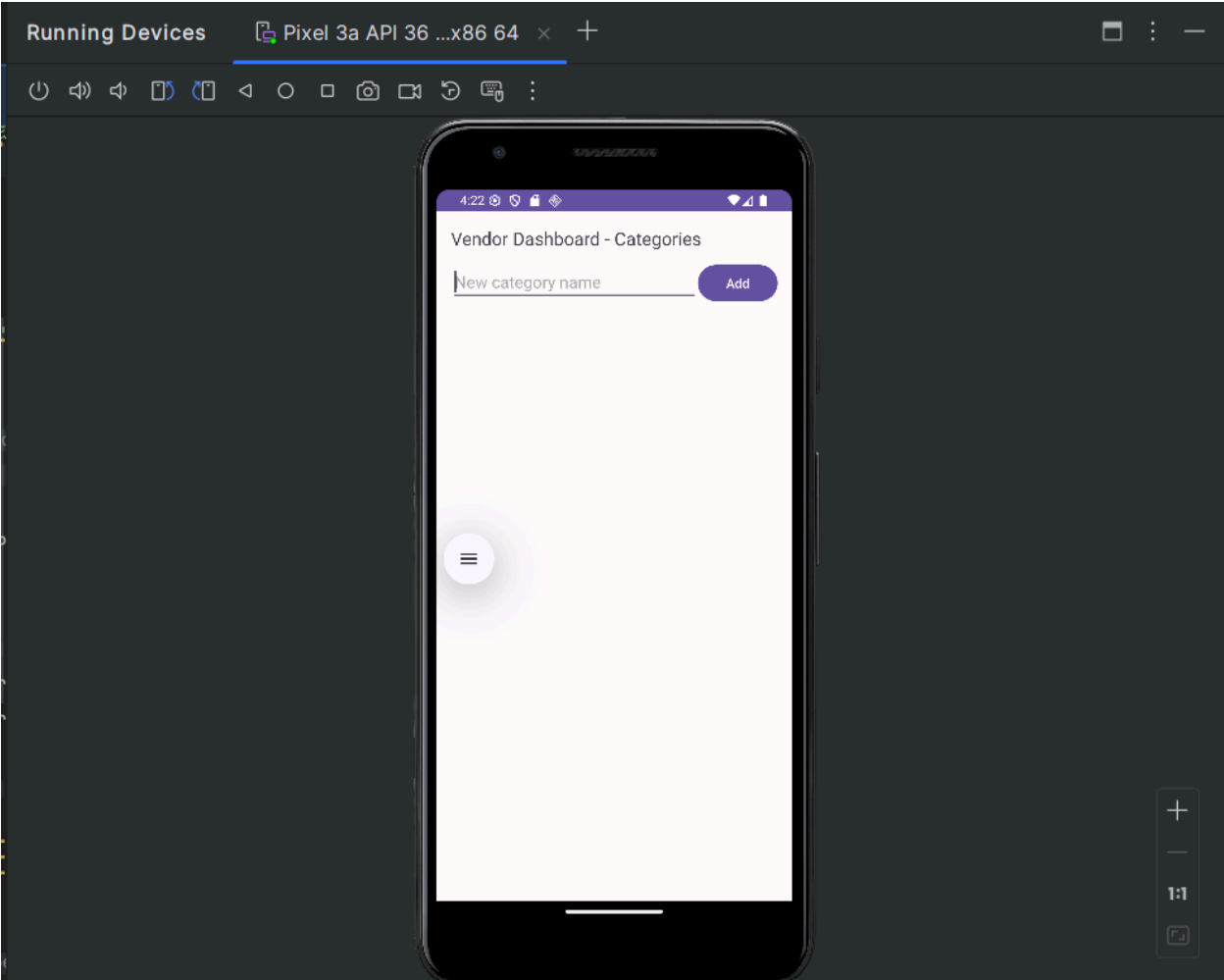
Testing results and screenshots:

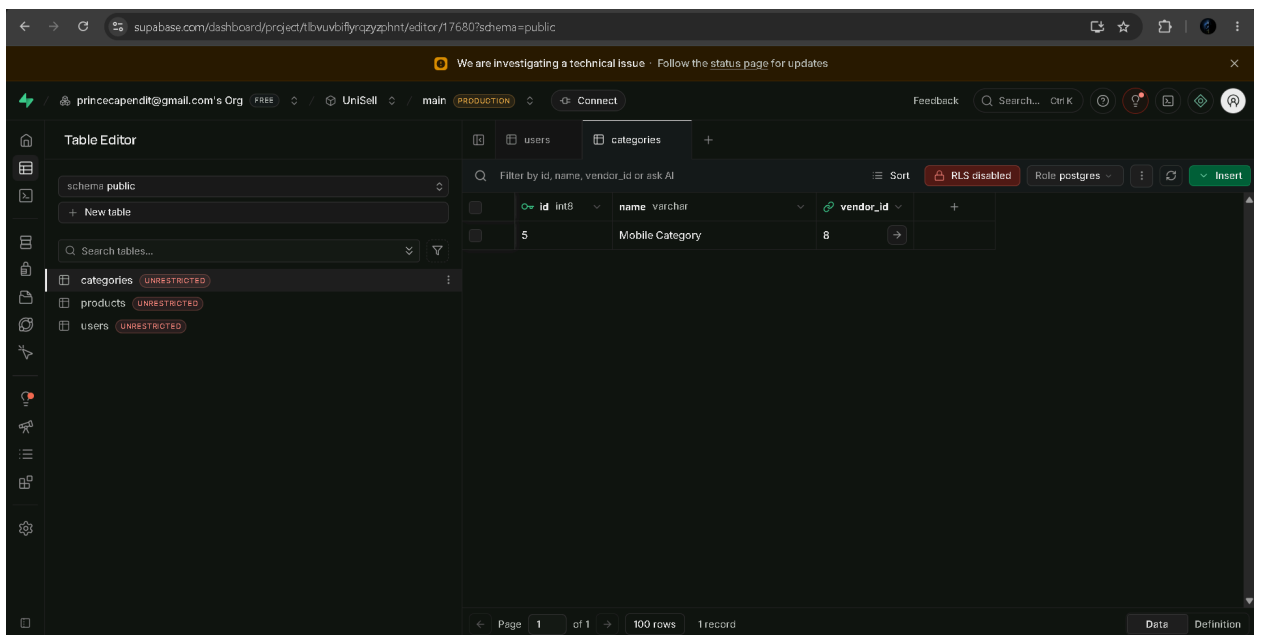
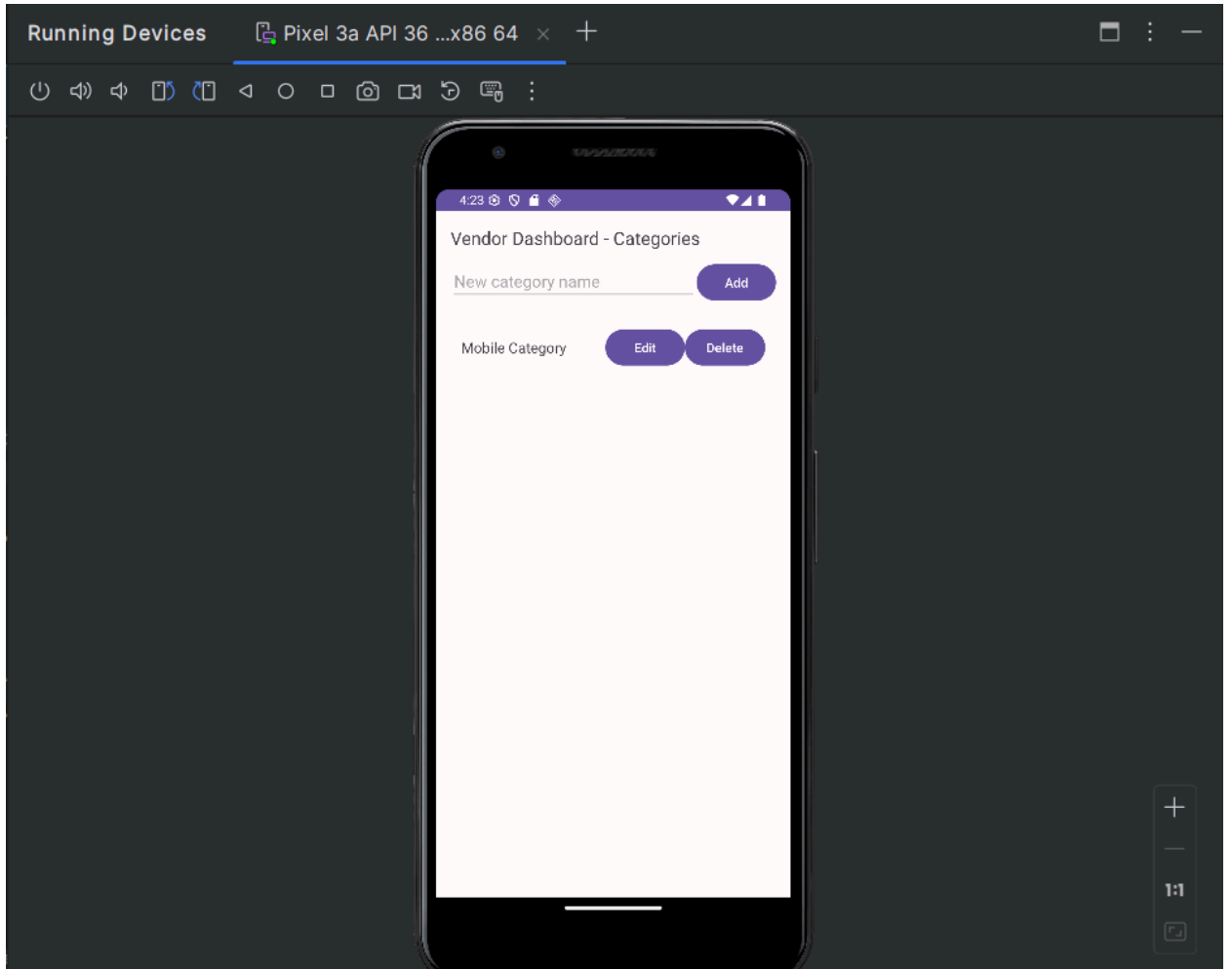


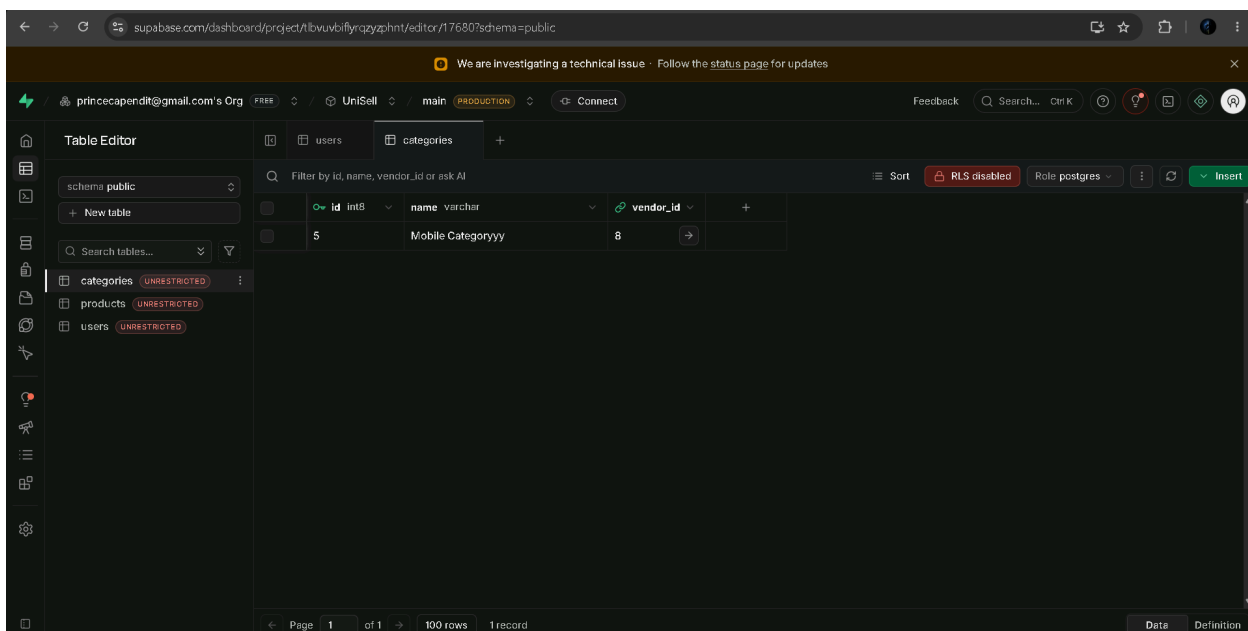
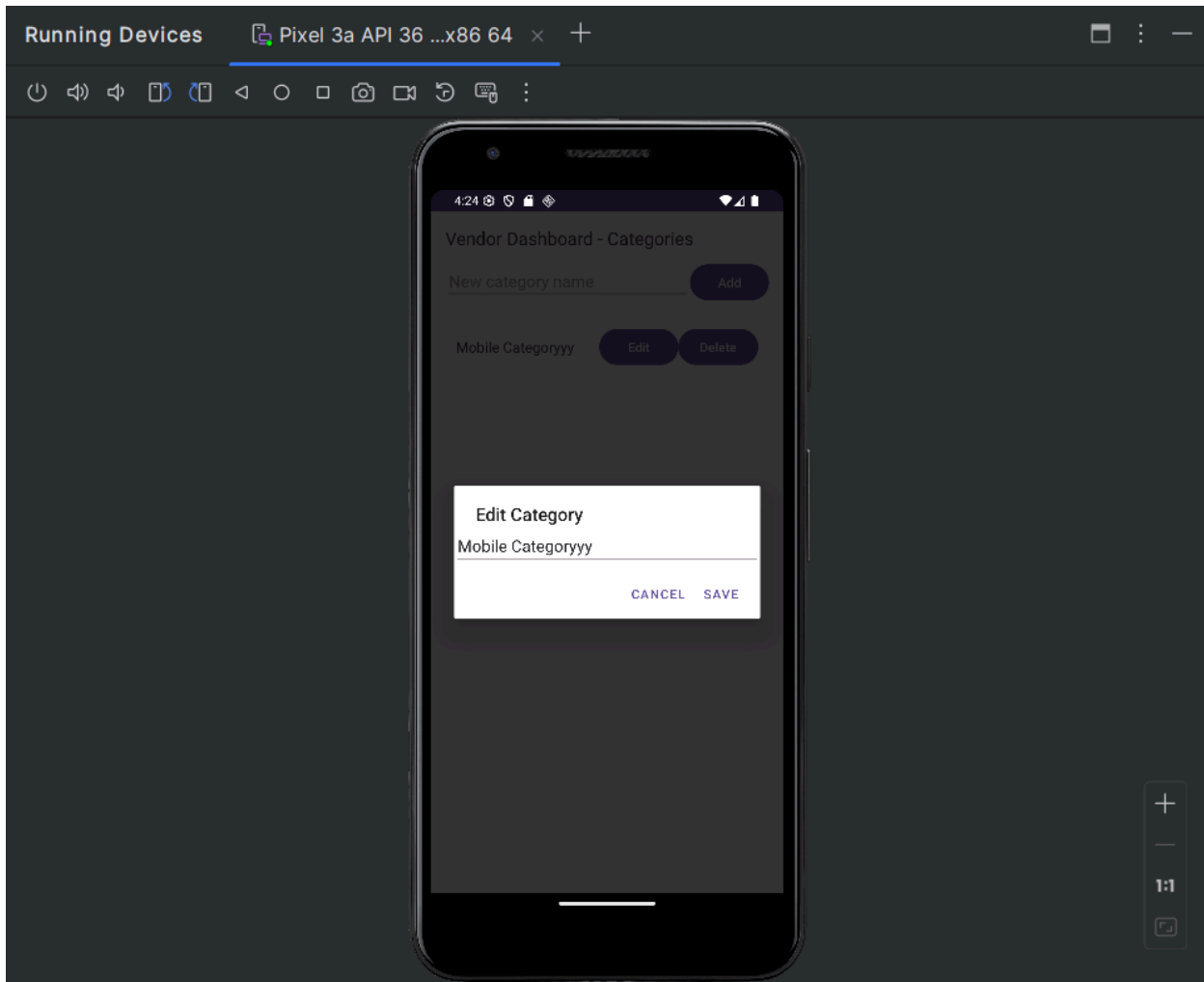


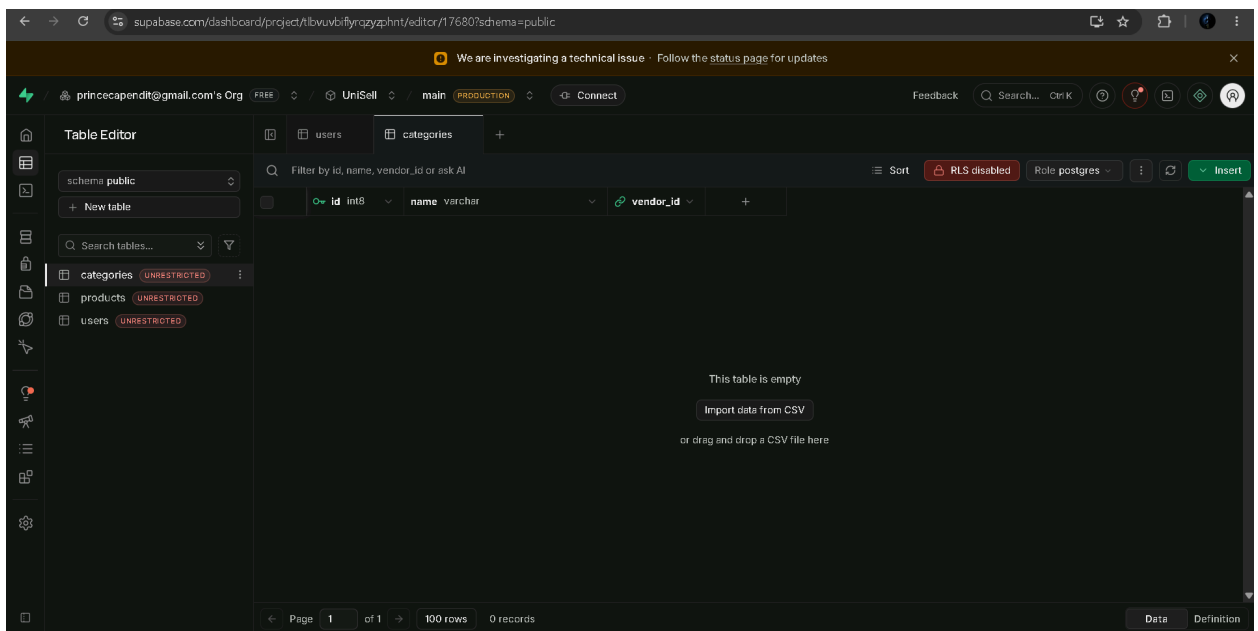
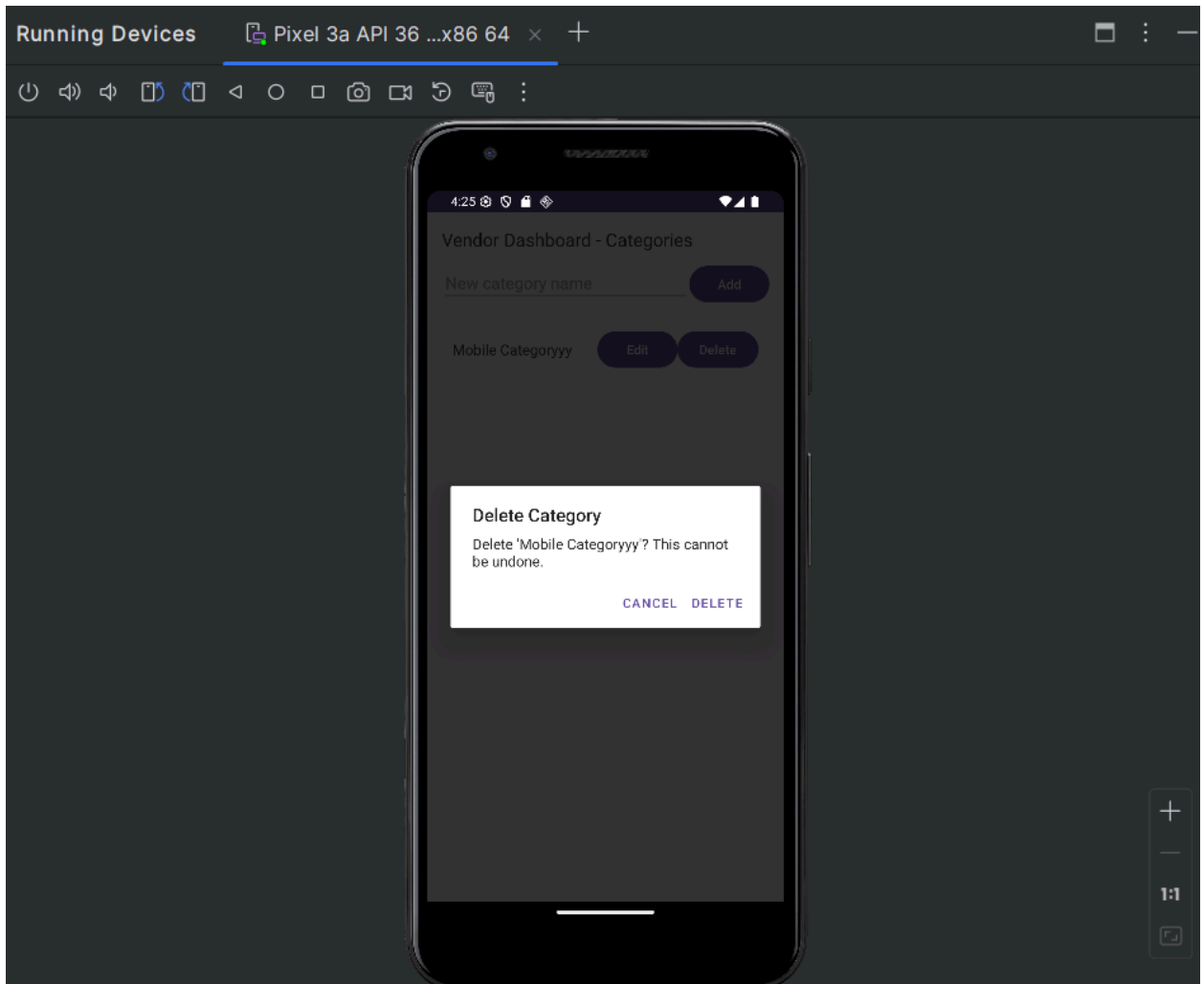


Mobile:

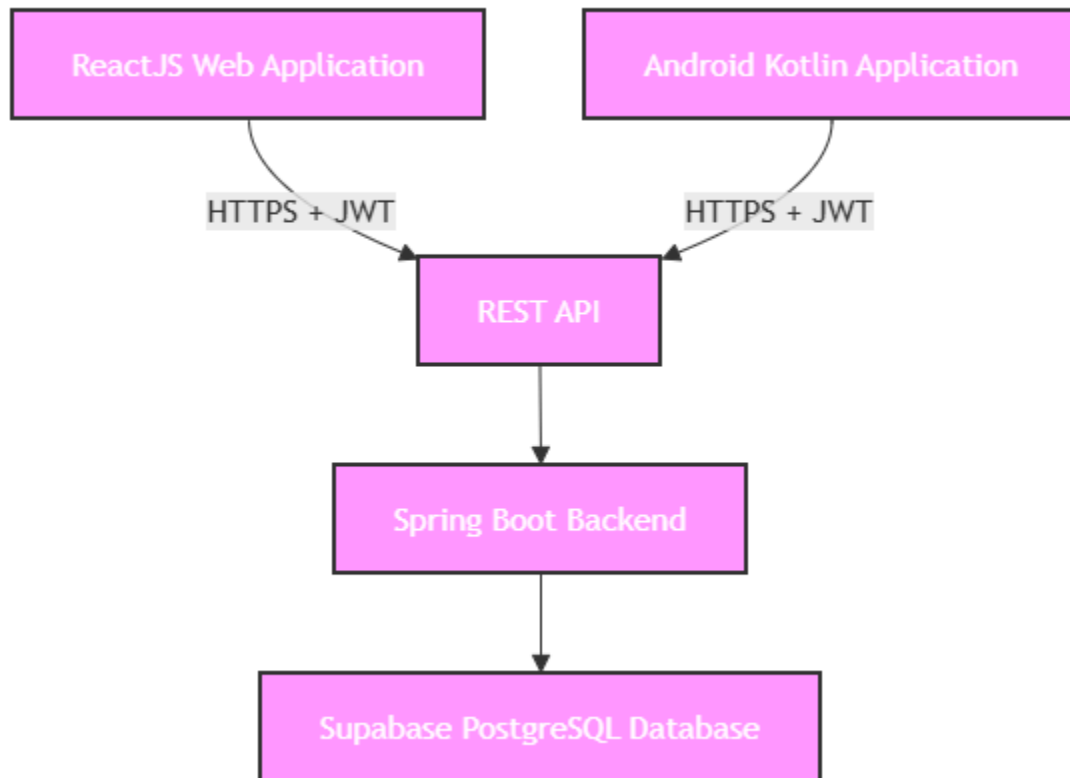








3. System Architecture Diagram



The web and the Android clients are simply frontends, and they never communicate with the database directly. Instead, only call the same set of REST endpoints. Each authenticated request includes a JWT in the Authorization header, and the JWT filter on the backend checks the token to see if it is valid and what role the caller is granted before the request passes through to any business logic, so that the vendor can only view his/her data. Any call made via REST is translated into JPA/Hibernate operations on the underlying Supabase Postgres database, and both web and Android read the same underlying rows, meaning the backend is the single source of truth.